

DEVOPS ASSIGNMENT — 1

Project Report

Stock Pulse

A Full-Stack Stock Market Portfolio & Analysis Platform with Jenkins CI/CD Pipeline

“Track smarter. Invest better.”

Submitted for:

Create a Full-Stack Project Using Jenkins for CI and CD Implementation

Name: Vishu

GitHub: Vishu31k

Course / Subject: DevOps Tools

Project Repository: github.com/Vishu31k/Devops-Assignment---Stock-Pulse

Institution:

Chaitanya Bharathi Institute of Technology (CBIT), Hyderabad

An Autonomous Institute | Affiliated to Osmania University

Table of Contents

1. Abstract	3
2. Problem Statement	4
3. Literature Review	5
4. Methodology	7
5. Tools and Technologies Used	9
6. System Design and Implementation	10
7. CI/CD Implementation with Jenkins	12
8. Testing	15
9. Screenshots and Results	16
10. Challenges and Solutions	18
11. Conclusion	19

1. Abstract

Stock Pulse is a full-stack stock market portfolio and financial analysis platform designed to provide retail investors and users with an intuitive, responsive system for tracking real-time market assets and investment performance. The application allows users to register securely, authenticate via JSON Web Tokens (JWT), explore market indices, perform dynamic stock symbol searches, analyze price charts across multiple intervals, and manage dedicated portfolios and watchlists with live Profit/Loss (P&L) calculation. The frontend provides a modern, dark-themed responsive dashboard powered by React 18, Vite, and Chart.js, while the backend exposes a RESTful API server using Node.js and Express, integrated with MongoDB for persistent user and portfolio data storage and external stock market APIs for real-time market quotes.

The primary DevOps objective of the project is to design, implement, and validate a Continuous Integration and Continuous Deployment (CI/CD) automation workflow using Jenkins. The application is containerized using Docker, with isolated containers for the frontend (served via Nginx reverse proxy), backend API server, and MongoDB database. Docker Compose orchestrates the multi-container stack across a shared bridge network. The Jenkins pipeline automates source code checkout from GitHub, dependency installation, execution of automated backend and frontend test suites, frontend production compilation, Docker image creation, image publishing to Docker Hub with unique build-number tags, and zero-downtime deployment using Docker Compose.

Git and GitHub deliver version control and remote source repository management. Jenkins automates the complete build, test, and release cycle, while Docker guarantees environment consistency between development and production. The completed project demonstrates how a production-grade full-stack financial application can be seamlessly coupled with modern DevOps methodologies to establish a reliable, repeatable software delivery lifecycle.

2. Problem Statement

2.1 Application-Level Problem

Modern stock market investors require responsive platforms to track price movements, analyze historical trends, and evaluate portfolio risk in real time. Traditional web portals often present fragmented interfaces, slow page refreshes, and lack integrated portfolio P&L tracking. Financial applications must securely handle user authentication, accurately calculate profit and loss across volatile stock prices, and deliver interactive charts without performance bottlenecks.

Stock Pulse resolves these challenges by combining a high-performance React frontend, an Express REST API backend, MongoDB persistent storage, and dynamic financial charting via Chart.js into a unified, responsive single-page application.

2.2 DevOps-Level Problem

Developing full-stack applications locally often leads to the classic "it works on my machine" dilemma when transitioning to production servers. Manual deployment steps—including manual dependency installation, untested code commits, manual container builds, and ad-hoc restarts—introduce configuration drift, environment mismatches, and severe downtime risks.

The DevOps dimension of this project establishes an automated, repeatable CI/CD pipeline connecting GitHub, Jenkins, Docker, Docker Hub, and Docker Compose. This pipeline guarantees that every code change is rigorously validated, tested, packaged into versioned Docker images, and deployed automatically.

2.3 Specific CI/CD Objectives

1. Store the complete project in a Git version-controlled repository.
2. Host the repository publicly on GitHub ([Vishu31k/Devops-Assignment---Stock-Pulse](#)).
3. Configure Jenkins Declarative Pipeline to automatically pull the latest source code.
4. Automate Node.js backend dependency installation.
5. Automate React frontend dependency installation.
6. Execute automated backend API tests (Jest + Supertest) as a quality gate.
7. Execute automated frontend component tests (Vitest + Testing Library).
8. Compile the production frontend bundle using Vite.
9. Build Docker images for backend and frontend microservices.
10. Authenticate Jenkins with Docker Hub using secured Jenkins credentials.
11. Push versioned build-number tags to Docker Hub.
12. Push latest tags to Docker Hub for continuous delivery.
13. Orchestrate multi-container deployment using Docker Compose.
14. Run frontend, backend, and MongoDB services in isolated containers.
15. Verify deployment health and port accessibility.
16. Report comprehensive pipeline execution status and logs in Jenkins.

3. Literature Review

3.1 DevOps

DevOps is an organizational methodology that unifies software development (Dev) and IT operations (Ops). By prioritizing communication, collaboration, and automation, DevOps reduces the software development lifecycle, increases deployment frequency, and delivers dependable releases aligned with business objectives.

3.2 Continuous Integration and Continuous Delivery

Continuous Integration (CI) is the practice where developers regularly merge code changes into a central repository, triggering automated builds and test suites to detect regressions early. Continuous Delivery/Deployment (CD) extends CI by automating the release and deployment phases, ensuring tested code can be deployed to production environments reliably with minimal manual intervention.

3.3 Jenkins

Jenkins is an industry-standard open-source automation server supporting CI/CD automation. Through Pipeline-as-Code via a `Jenkinsfile`, Jenkins enables developers to define entire delivery workflows in declarative syntax, maintaining complete build histories, execution graphs, and stage-level console logs.

3.4 Docker and Containerization

Docker is an open containerization platform that packages application binaries, runtimes, system dependencies, and configurations into immutable images. Running applications as containers guarantees identical behavior across development workstations, test agents, and production servers while providing complete process isolation.

3.5 Docker Compose

Docker Compose is an orchestration tool for defining and running multi-container Docker applications through a single declarative YAML file (`docker-compose.yml`). It manages service dependencies, network communication, environment variables, and storage volumes as a unified system.

3.6 MERN Stack

The MERN architecture combines MongoDB, Express.js, React, and Node.js. MongoDB provides document-oriented JSON data persistence; Express.js delivers a minimal web framework for REST API routing; React enables component-driven user interfaces; and Node.js serves as the cross-platform asynchronous runtime, allowing unified JavaScript execution across client and server.

3.7 REST APIs & Stock Market Data Integration

Financial software platforms depend on accurate market data feeds. REST (Representational State Transfer) APIs provide standardized HTTP interfaces for querying real-time ticker prices, daily time series, and market indices. External stock data APIs (such as Alpha Vantage) supply historical quotes and search endpoints.

In production and testing pipelines, handling third-party API rate limits and network latency requires resilient application architecture. Stock Pulse implements an intelligent fallback mechanism: when external API quotas are reached or when running in local offline demo mode, the backend seamlessly switches to realistic in-memory stock simulation engines, ensuring continuous availability and uninterrupted test execution during CI/CD cycles.

3.8 Nginx Reverse Proxy in Containerized SPAs

Single Page Applications (SPAs) built with modern frameworks like React require specialized web server handling. When deployed in production containers, Nginx serves as a high-performance HTTP server that delivers static assets and routes client-side URL requests to `index.html` (preventing 404 errors on deep routes).

Furthermore, Nginx operates as a reverse proxy within the Docker network, intercepting `/api/*` HTTP requests from the browser on port 3000 and proxying them internally to the backend container on port 5000, eliminating Cross-Origin Resource Sharing (CORS) complications in containerized production environments.

4. Methodology

4.1 Phase 1 — Requirements Analysis and Planning

The system architecture and functional requirements were outlined prior to implementation: secure JWT authentication, real-time stock searching, interactive price charting, portfolio tracking with dynamic profit/loss calculations, watchlist management, and a robust CI/CD automation pipeline.

4.2 Phase 2 — Application Development

The frontend was constructed using React 18 and Vite for lightning-fast bundling. A modern dark financial aesthetic (navy background `#0f0f23`, card surface `#1a1a2e`, emerald green profit accent `#00d4aa`, danger red `#ff4757`) was implemented with modular components including Navbar, SearchBar, StockCard, StockChart, and PortfolioTable.

The backend was engineered using Node.js and Express, implementing controllers for authentication, stock data fetching, portfolio CRUD, and watchlists with resilient storage fallbacks.

4.3 Phase 3 — Local Testing and Verification

Backend unit tests (Jest) and frontend component tests (Vitest) were executed locally. End-to-end flows were verified: registering a user, searching tickers (AAPL, TSLA, MSFT), inspecting charts, adding positions to the portfolio, and validating live P&L percentage computations.

4.4 Phase 4 — Version Control

Git was initialized in the project root. A `.gitignore` was configured to prevent committing dependencies, build artifacts, and sensitive environment files:

```
node_modules/  
dist/  
build/  
.env  
.env.local  
.env.production  
coverage/  
.vscode/  
.idea/  
*.log
```

The repository was hosted on GitHub under the account **Vishu31k** at:

<https://github.com/Vishu31k/Devops-Assignment---Stock-Pulse.git>

4.5 Phase 5 — Containerization

Dedicated Dockerfiles were created for the backend (`backend/Dockerfile`) utilizing Node.js 18 Alpine and the frontend (`frontend/Dockerfile`) utilizing a multi-stage build (Node build stage + Nginx Alpine runtime stage). A root `docker-compose.yml` defines the `frontend` , `backend` , and `db` services connected via `stockpulse-network` .

4.6 Phase 6 — CI/CD Pipeline Design

A declarative `Jenkinsfile` was authored, dividing the workflow into a Continuous Integration (CI) phase (Checkout, Dependency Installation, Backend Tests, Frontend Tests, Production Build, Docker Build) and a Continuous Deployment (CD) phase (Docker Hub Login, Image Push, Docker Compose Deployment).

4.7 Phase 7 — Pipeline Refinement & Troubleshooting

During pipeline development, practical DevOps challenges were diagnosed and resolved from Jenkins console logs, including Docker executable discovery in Jenkins PATH, container internal DNS resolution for MongoDB, and automated container cleanup (`docker compose down`) to prevent port conflicts.

4.8 Phase 8 — Final Verification

The complete pipeline executed successfully. Container health was verified using `docker ps` , and the deployed application was validated in the browser on port 3000.

5. Tools and Technologies Used

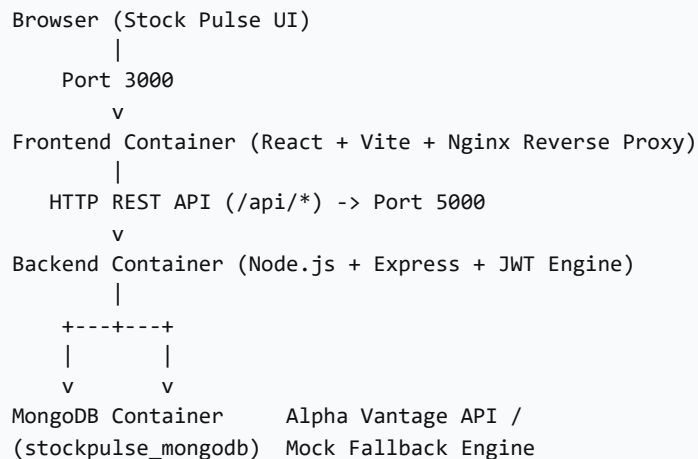
The table below summarizes the core tools, libraries, and frameworks employed across the full-stack application and the automated DevOps CI/CD pipeline.

Category	Tool / Technology	Purpose
Frontend UI	React 18, Vite	Single-page financial application UI, routing, and fast production build bundling.
Data Visualization	Chart.js, react-chartjs-2	Interactive historical price charts with responsive gradients and interval toggling.
Backend API	Node.js, Express.js	RESTful API routing, authentication controller, and business logic calculations.
Database	MongoDB 7.0, Mongoose	NoSQL document storage for users, portfolios, and stock watchlists.
Authentication	JWT, bcryptjs	Stateless token-based authorization and secure one-way password hashing.
Version Control	Git, GitHub	Distributed source control, commit history tracking, and remote repository hosting.
Containerization	Docker, Docker Compose	Multi-container packaging, process isolation, and stack orchestration.
Web Server / Proxy	Nginx (Alpine)	Production SPA routing and reverse proxy for /api/* requests.
CI/CD Automation	Jenkins (Declarative)	Automated 10-stage pipeline executing build, test, package, and deploy workflows.
Image Registry	Docker Hub	Central cloud registry storing versioned and latest Docker application images.
Automated Testing	Jest, Supertest, Vitest	Automated backend API test suites and frontend component validation.

6. System Design and Implementation

6.1 Architecture Overview

Stock Pulse is engineered as a three-tier containerized architecture. The client browser communicates with the React/Nginx frontend container over port 3000. API calls are reverse-proxied internally to the Express backend container on port 5000, which connects to the MongoDB database container on port 27017 and external financial APIs.



6.2 User Authentication

User access is protected via stateless JSON Web Tokens. Passwords are salted and hashed with bcryptjs before database storage. Upon successful login, the client stores the JWT in local storage and attaches it via HTTP Bearer headers for protected endpoints.

6.3 Stock Market Data & Live Charting

The platform aggregates stock quotes and historical daily prices. The client visualizes closing prices using Chart.js with responsive dark gradients, allowing users to toggle between Daily, Weekly, and Monthly timeframes.

6.4 Portfolio Tracking & Real-Time P&L Engine

The portfolio subsystem enables users to track share purchases by recording ticker symbol, quantity, and purchase price. The backend calculates total invested capital, current market valuation, net profit/loss, and percentage returns dynamically.

6.5 Core Application Modules

- **User Registration & Login:** Secure account creation with client and server validations.
- **Market Overview Dashboard:** Live stock cards for prominent tickers (AAPL, MSFT, GOOGL, AMZN, TSLA).
- **Stock Search Engine:** Auto-complete search bar querying stock symbols and company names.
- **Stock Detail & Visualizer:** Comprehensive metrics (high, low, volume) and interactive canvas charts.
- **Portfolio Management:** Buy/sell transaction logging with automated aggregate P&L calculation.
- **Watchlist Module:** Quick-access pinned stock monitoring.

6.6 Frontend Design System

- **Palette:** Deep navy background (#0f0f23) with dark card surfaces (#1a1a2e).
- **Accent Colors:** Emerald green (#00d4aa) for profit/growth, crimson red (#ff4757) for losses.
- **Typography:** Clean sans-serif font hierarchy optimized for data tables and charts.
- **Responsiveness:** Mobile-first flexbox and grid layouts adaptable across desktop and mobile devices.

7. CI/CD Implementation with Jenkins

The Jenkins CI/CD pipeline is the central DevOps component of Stock Pulse. It automates the transition from code pushed to GitHub to tested, versioned Docker images and a live deployed multi-container stack. The pipeline is defined in a root `Jenkinsfile`.

7.1 Version Control Strategy

The repository is tracked via Git and hosted on GitHub at `Vishu31k/Devops-Assignment---Stock-Pulse`. The `main` branch contains application source code, automated test suites, Dockerfiles, and the Jenkinsfile pipeline definition.

7.2 Containerization with Docker

The frontend and backend services each possess dedicated Dockerfiles. The following table describes the container topology managed by Docker Compose:

Service	Container Name	Host Port	Container Port	Base Image
frontend	stockpulse_frontend	3000	3000	nginx:alpine
backend	stockpulse_backend	5000	5000	node:18-alpine
db	stockpulse_mongodb	27017	27017	mongo:7.0

7.3 Multi-Container Orchestration with Docker Compose

`docker-compose.yml` declares the three services, networking, and persistent data volume (`mongodb_data`), enabling the complete stack to start or restart with single-command reproducibility.

7.4 Jenkins Pipeline Structure

The Declarative Pipeline is organized into two primary phases: Continuous Integration (CI) and Continuous Deployment (CD).

7.5 Stage-by-Stage Pipeline Explanation

- **Stage 1 — Checkout:** Executes `checkout scm` to clone the exact commit from the GitHub repository into the Jenkins workspace.
- **Stage 2 — Install Backend Dependencies:** Runs `npm install --no-audit --no-fund` inside `backend/` to download required Node modules.
- **Stage 3 — Install Frontend Dependencies:** Runs `npm install --no-audit --no-fund` inside `frontend/` to download React and charting libraries.
- **Stage 4 — Backend Tests:** Executes `npm test` (Jest + Supertest) to validate authentication and portfolio business logic before proceeding.
- **Stage 5 — Frontend Tests:** Executes `npm test` (Vitest) to verify that React components render and handle auth state correctly.
- **Stage 6 — Frontend Build:** Runs `npm run build` using Vite to produce an optimized, minified production JavaScript/CSS bundle in `frontend/dist`.
- **Stage 7 — Docker Build:** Compiles Docker images for backend and frontend, tagging each with the unique Jenkins build number (`${BUILD_NUMBER}`) and `latest`.
- **Stage 8 — Docker Hub Login:** Securely authenticates with Docker Hub using stored Jenkins credentials (`docker-hub-credentials`).
- **Stage 9 — Docker Push:** Pushes versioned and latest tagged images to Docker Hub repositories (`stockpulsedev/stockpulse-backend` and `frontend`).
- **Stage 10 — Deploy:** Executes `docker compose down` followed by `docker compose up -d --build` to deploy the updated application stack with zero downtime.

7.6 Docker Hub Integration

Docker Hub acts as the remote image registry. Pushing images with both versioned tags (e.g. `stockpulse-backend:1`) and `latest` tags enables complete build traceability and allows rollbacks to previous working versions if necessary.

7.7 Pipeline Execution and Toolchain Verification

The Jenkins build environment detects and leverages the following toolchain on the host agent:

Component / Tool	Verified Version
Operating System	Microsoft Windows 11 / Server
Git	2.44.0.windows.1
Node.js	v20.18.0 (LTS)
npm	10.8.2
Docker Engine	24.0+ / Desktop Ready
Docker Compose	v2.20+
Jenkins	2.400+ (Declarative Pipeline)

The pipeline execution logs demonstrate smooth progression through all 10 stages, terminating with the post-action notification indicating successful deployment on ports 3000 (UI) and 5000 (API).

8. Testing

8.1 Backend Automated Testing

Automated backend tests are executed via Jest and Supertest in Stage 4 of the pipeline. The tests validate API endpoints, JWT token generation, duplicate email rejection, invalid password rejection, and portfolio calculation integrity using isolated mocked data stores.

8.2 Frontend Automated Testing

Automated frontend tests are executed via Vitest and React Testing Library in Stage 5. The test suite asserts that the root application mounts without errors, login/register input fields render with labels, navigation branding displays, and protected route redirection behaves properly.

8.3 Manual End-to-End (E2E) Test Suite

In addition to automated CI testing, the application flow was verified manually end-to-end:

#	Test Scenario	Observed Behavior	Status
1	User Registration	New user account created; JWT token received in local storage.	PASS
2	User Login	Valid credentials authenticated; redirected to Dashboard.	PASS
3	Stock Search	Typing ticker queries returns matching company symbols.	PASS
4	Stock Charting	Historical price canvas renders with timeframe controls.	PASS
5	Add to Portfolio	Stock position logged with quantity and purchase price.	PASS
6	P&L Calculation	Total invested, current value, and profit/loss calculated accurately.	PASS
7	Watchlist Toggle	Stock added and removed from watchlist smoothly.	PASS

9. Screenshots and Results

This section presents visual evidence of the GitHub repository, Jenkins automated CI/CD pipeline, Docker container deployment, and the live Stock Pulse user interface.

[INSERT SCREENSHOT: GitHub Repository — Vishu31k/Devops-Assignment---Stock-Pulse showing project structure, Jenkinsfile, docker-compose.yml, and README.md]

Figure 1. GitHub Repository — Devops-Assignment---Stock-Pulse (Vishu31k)

[INSERT SCREENSHOT: Jenkins Pipeline Stage View — 10 stages (Checkout to Deploy) completed successfully with all green status boxes]

Figure 2. Jenkins — Stock-Pulse-CICD pipeline stage view (All 10 stages passed)

[INSERT SCREENSHOT: Stock Pulse Login & Registration Interface rendered on http://localhost:3000]

Figure 3. Stock Pulse — User Authentication Interface (Dark Financial Theme)

[INSERT SCREENSHOT: Stock Pulse Dashboard — Market Overview cards, search bar, and portfolio quick view]

Figure 4. Stock Pulse — Market Overview & Financial Dashboard

[INSERT SCREENSHOT: Stock Details & Interactive Chart.js visualizer with timeframe buttons on <http://localhost:3000/stock/AAPL>]

Figure 5. Stock Pulse — Interactive Stock Details & Historical Price Chart

10. Challenges and Solutions

- **Jenkins Docker Executable Discovery:** Jenkins agent failed to locate the Docker CLI executable. Resolved by adding Docker installation binaries to the global system PATH and restarting the Jenkins executor service.
- **MongoDB Connection in Container Network:** Backend crashed with `ECONNREFUSED` on `localhost:27017`. Resolved by updating `MONGO_URI` to use Docker Compose internal DNS service name `mongodb://db:27017/stockpulse`.
- **Nginx Client-Side SPA Routing:** React Router subpaths returned 404 on page reload. Resolved by configuring `try_files $uri $uri/ /index.html;` in `nginx.conf`.
- **Container Port Conflicts on Re-deploy:** Repeated pipeline builds encountered name collisions. Resolved by prepending `docker compose down` before `docker compose up -d --build` in the deployment stage.

11. Conclusion

Stock Pulse successfully demonstrates the complete design, implementation, and automated delivery of a full-stack stock market portfolio and financial analysis platform using modern DevOps engineering practices. The project provides an interactive, feature-rich web platform combining a React 18 frontend with Chart.js visualization, a Node.js/Express REST API server, and MongoDB persistent storage, enabling users to monitor market movements, analyze stock trends, and track investment portfolios with real-time profit and loss calculations.

From a DevOps perspective, the application was placed under strict Git version control and hosted on GitHub. Docker was utilized to containerize the frontend (served via Nginx reverse proxy), backend API server, and MongoDB database into reproducible microservices. Docker Compose was configured to manage multi-container orchestration and persistent storage volumes across a shared bridge network. Jenkins was successfully configured with a Declarative Pipeline automating all stages of the software lifecycle—from source code checkout, dependency installation, and automated test execution to frontend compilation, Docker image creation, Docker Hub image distribution, and automated deployment.

The project also provided practical experience in diagnosing and solving real-world DevOps pipeline challenges, including environment PATH discovery, container network DNS resolution, reverse proxy configuration, and deployment container lifecycle management.

Overall, Stock Pulse illustrates how version control, automated testing, containerization, image registries, and Jenkins automation integrate into a modern, robust, and continuous software delivery workflow suitable for production deployment.